

รายงาน

เรื่อง การประยุกต์ใช้โครงสร้างข้อมูล ในการพัฒนาเกม Aetheris

(โปรเจกต์ Aetheris)

จัดทำโดย

กลุ่ม MTG (MorThodGame)

1. เธียรภาส ธนะสีลังกูร รหัสนักศึกษา 682110122
2. จิรายุ โพธิ์ทอง รหัสนักศึกษา 682110107
3. อรอักษรา อักษรณรงค์ รหัสนักศึกษา 682110153
4. พงศภัค ท่าดีสม รหัสนักศึกษา 682110130

เสนอ

อาจารย์ ดร.เกียรติกร สำอางค์กูล

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 958211 Data Structure for Game Development

ภาคเรียนที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยเชียงใหม่

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชาโครงสร้างข้อมูล จัดทำขึ้นเพื่ออธิบายโครงสร้างข้อมูลที่กลุ่มนำมาใช้และวางแผนจะนำมาใช้ในการพัฒนาเกม Aetheris (ชื่อโปรเจกต์ Aetheris) ซึ่งเป็นเกมแนว RPG ที่ผู้เล่นพาทีมสี่คนเข้าไปสำรวจด่าน ต่อสู้แบบผลัดตา เก็บของ และเลือกว่าจะถอนตัวกลับหรือเสี่ยงเดินต่อ เกมเขียนด้วยภาษา C# บน .NET 9 และใช้ MonoGame 3.8

เนื้อหาแบ่งเป็น 5 บท ได้แก่ บทนำ ทฤษฎีที่เกี่ยวข้อง การออกแบบระบบ การนำโครงสร้างข้อมูลไปใช้ในเกม และบทสรุป ตัวอย่างโค้ดและไดอะแกรมทั้งหมดมาจากโค้ดจริงของโปรเจกต์ ท้ายเล่มมีภาคผนวกรวบรวมโค้ดเพิ่มเติมและรายการโครงสร้างข้อมูลทั้งหมดที่ใช้

คณะผู้จัดทำหวังว่ารายงานนี้จะเป็นประโยชน์แก่ผู้อ่าน หากมีข้อผิดพลาดประการใดขออภัยมา ณ ที่นี้

คณะผู้จัดทำ
กลุ่ม MTG

สารบัญ

หน้า

คำนำ **2**

สารบัญ **3**

สารบัญภาพ **5**

สารบัญตาราง **6**

บทที่ 1 บทนำ **7**

1.1 ที่มาและความสำคัญ 7

1.2 วัตถุประสงค์ 7

1.3 ขอบเขต 7

1.4 ภาพรวมของเกม Aetheris 7

1.5 ประโยชน์ที่คาดว่าจะได้รับ 8

1.6 นิยามศัพท์ 8

บทที่ 2 ทฤษฎีที่เกี่ยวข้อง **9**

2.1 การวิเคราะห์ประสิทธิภาพ 9

2.2 อาร์เรย์และลิสต์ 9

2.3 สแตกและคิว 9

2.4 ตารางแฮช 9

2.5 กราฟ ต้นไม้ และการค้นหาแนวกว้าง 10

2.6 บิตมาสก์และการเรียงลำดับหลายคีย์ 10

บทที่ 3 การออกแบบระบบ **11**

3.1 โครงสร้างของโปรแกรม 11

3.2 เกณฑ์การเลือกโครงสร้างข้อมูล 11

3.3 แผนภาพคลาส 12

บทที่ 4 การนำโครงสร้างข้อมูลไปใช้ในเกม **14**

4.1 สแตกของหน้าจอ 14

4.2 แผนที่ด่านแบบกราฟ 15

4.3 ผังห้องแบบอาร์เรย์สองมิติ 17
4.4 การหาเส้นทางด้วย BFS 17
4.5 แถวการรบและบิตมาสก์ของตำแหน่ง 19
4.6 ลำดับการกระทำและคิวเหตุการณ์ 21
4.7 สถานะผิดปกติและ AI ของศัตรู 22
4.8 ข้อมูลเนื้อหาและกราฟบทสนทนา 23
4.9 แคชทรัพยากรและแฟล็ก 24
4.10 การบันทึกเกม 24
4.11 โครงสร้างข้อมูลที่วางแผนจะใช้ 24
บทที่ 5 สรุปผลและข้อเสนอแนะ 26
5.1 สรุปผล 26
5.2 ปัญหาที่พบ 26
5.3 ข้อเสนอแนะ 26
บรรณานุกรม 28
ภาคผนวก ก โค้ดการสร้างแผนที่ 29
ภาคผนวก ข รายการโครงสร้างข้อมูลในโปรเจกต์ 31
ภาคผนวก ค ตัวอย่างไฟล์บันทึกเกม 32

สารบัญภาพ

หน้า

ภาพที่ 3.1 โครงสร้างของโปรแกรมและโครงสร้างข้อมูลหลักในแต่ละส่วน 11
ภาพที่ 3.2 แผนภาพคลาส UML ของคลาสหลัก 13
ภาพที่ 4.1 การ Push และ Pop หน้าจอการต่อสู้ 14
ภาพที่ 4.2 ตารางแฮชของห้องและรายการประชิดภายในห้อง 15
ภาพที่ 4.3 ตัวอย่างแผนที่ Trial เส้นทึบคือเส้นของต้นไม้ เส้นประคือทางวน 16
และ d คือความลึกจาก BFS
ภาพที่ 4.4 คิว ตารางช่องก่อนหน้า และการย้อนรอยสร้างเส้นทาง 18
ภาพที่ 4.5 ผังห้อง Plaza ใน Tile[21,12] พร้อมระยะทาง BFS และเส้นทางที่ 19
สั้นที่สุด
ภาพที่ 4.6 อาร์เรย์แถวการรบ การสลับตำแหน่ง และช่องว่างเมื่อมีตัวตาย 20
ภาพที่ 4.7 การเก็บเซตของตำแหน่งด้วยบิตมาสก์ 21
ภาพที่ 4.8 ลำดับการทำงานในหนึ่งเทิร์น 22
ภาพที่ 4.9 การสุ่มเป้าหมายแบบถ่วงน้ำหนัก 23
ภาพที่ 4.10 กราฟบทสนทนาของ NPC ชื่อ Maren 23
ภาพที่ 4.11 การแคชทรัพยากรด้วย Dictionary 24

สารบัญตาราง

หน้า

ตารางที่ 1.1 นิยามศัพท์ที่ใช้ในรายงาน 8
ตารางที่ 2.1 ความซับซ้อนของการทำงานหลัก (กรณีเฉลี่ย) 10
ตารางที่ 3.1 ระบบในเกมและโครงสร้างข้อมูลที่ใช้ 11
ตารางที่ 4.1 โครงสร้างข้อมูลที่วางแผนจะนำมาใช้ 24
ตารางที่ 5.1 สรุปโครงสร้างข้อมูลที่ใช้ในเกม 26
ตารางที่ ข.1 ฟิลด์ที่เป็นโครงสร้างข้อมูลในโค้ด 31

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

เกมต้องทำงานแบบเรียลไทม์ Aetheris แสดงผลที่ 60 เฟรมต่อวินาที แต่ละเฟรมจึงมีเวลาประมาณ 16.7 มิลลิวินาทีสำหรับรับอินพุต อัปเดตตัวละครและศัตรู และวาดภาพทั้งหมด ถ้าเลือกวิธีเก็บข้อมูลไม่ดี เช่น ค้นหาแบบไล่ทีละตัวในรายการยาว ๆ ทุกเฟรม เกมจะกระตุกได้ง่าย

นอกจากเรื่องความเร็ว โครงสร้างข้อมูลที่ตรงกับกฎของเกมยังทำให้โค้ดอ่านง่ายและเกิดบั๊กน้อยลง เช่น เอกสารออกแบบกำหนดว่าแถวการรบมี 4 ช่อง และเมื่อมีตัวละครตายช่องนั้นจะว่าง การใช้อาร์เรย์ขนาด 4 ที่ช่องว่างเป็น `null` ก็ตรงกับกฎนี้โดยตรง

1.2 วัตถุประสงค์

1. เพื่ออธิบายโครงสร้างข้อมูลที่ใช้ในเกม Aetheris พร้อมไดอะแกรมและโค้ดประกอบ
2. เพื่อให้เหตุผลว่าทำไมแต่ละระบบจึงเลือกใช้โครงสร้างข้อมูลนั้น
3. เพื่อวิเคราะห์ความซับซ้อนเชิงเวลาของการทำงานหลัก
4. เพื่อนำเสนอโครงสร้างข้อมูลที่วางแผนจะใช้ในอนาคต

1.3 ขอบเขต

รายงานนี้ศึกษาจากโค้ดของโปรเจกต์ Aetheris บนกิ่ง `dev` ประมาณ 60 ไฟล์ C# โดยไม่รวมโค้ดเก่าในโฟลเดอร์ `_Archive` ครอบคลุมระบบหน้าจอ แผนที่ด่าน ผังห้อง การหาเส้นทาง การต่อสู้ AI ของศัตรู บทสนทนา การโหลดทรัพยากร และการบันทึกเกม

1.4 ภาพรวมของเกม Aetheris

ผู้เล่นเริ่มที่ฐาน (Hub) เลือกตัวละคร 4 คน จัดแถว ใส่อุปกรณ์ และอัปเกรดค่าสถานะ จากนั้นเข้า Trial ซึ่งเป็นด่านที่มีห้องหลายห้องเชื่อมกันและสุ่มใหม่ทุกครั้ง ในห้องอาจมีศัตรู หีบสมบัติ จุดพัก เหตุการณ์ หรือจุดถอนตัว เมื่อเดินชนศัตรูจะเข้าสู่การ

ต่อสู้ ซึ่งผู้เล่นเลือกการกระทำให้ทุกคนก่อน แล้วการกระทำของทั้งสองฝ่ายจะทำงานตามลำดับความเร็ว ของที่เก็บได้ใน Trial จะเป็นของผู้เล่นจริงก็ต่อเมื่อถอนตัวสำเร็จ ถ้าแพ้ของทั้งหมดจะหายไป

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. เข้าใจการนำโครงสร้างข้อมูลไปใช้กับโปรแกรมที่ทำงานแบบเรียลไทม์
2. เปรียบเทียบข้อดีข้อเสียของโครงสร้างข้อมูลแต่ละแบบก่อนเลือกใช้ได้
3. มีเอกสารอ้างอิงสำหรับสมาชิกในกลุ่มเมื่อพัฒนาเกมต่อ

1.6 นิยามศัพท์

ตารางที่ 1.1 นิยามศัพท์ที่ใช้ในรายงาน

คำศัพท์	**ความหมาย**
Trial	ด่านหนึ่งด่าน ประกอบด้วยห้องหลายห้องที่เชื่อมกัน
Room	ห้องหนึ่งห้องใน Trial มีประเภท ผังห้อง ทางเชื่อม และสถานะ
Slot	ตำแหน่งในแถวการรบ 4 ช่อง: Front 1, Front 2, Back 1, Back 2
Combatant	ตัวละครหรือศัตรูหนึ่งตัวในการต่อสู้
Extraction	การถอนตัวออกจาก Trial เพื่อเก็บของที่ได้
Screen	สถานะหนึ่งของเกม เช่น เมนู ฐาน การสำรวจ การต่อสู้

บทที่ 2
ทฤษฎีที่เกี่ยวข้อง

2.1 การวิเคราะห์ประสิทธิภาพ

ประสิทธิภาพของการทำงานกับข้อมูลวัดด้วยสัญลักษณ์ Big-O ซึ่งบอกว่าเวลาที่ใช้เพิ่มขึ้นอย่างไรเมื่อข้อมูลมีขนาด n เช่น O(1) ใช้เวลาคงที่ O(n) เพิ่มตามจำนวนข้อมูล และ O(n log n) คือเวลาของการเรียงลำดับที่ดีที่สุดแบบเปรียบเทียบ บางการทำงานคิดแบบเฉลี่ย (amortized) เช่น การเพิ่มข้อมูลท้าย List ที่บางครั้งต้องขยายอาร์เรย์ แต่โดยเฉลี่ยยังเป็น O(1) สำหรับเกมยังต้องคำนึงถึงการจองหน่วยความจำด้วย เพราะ .NET มี Garbage Collector ที่อาจทำให้เกมสะดุดถ้าสร้างวัตถุชั่วคราวมากเกินไป

2.2 อาร์เรย์และลิสต์

อาร์เรย์ (`T[]`) เก็บข้อมูลต่อเนื่องในหน่วยความจำ เข้าถึงด้วยดัชนีได้ใน O(1) แต่มีขนาดคงที่ อาร์เรย์สองมิติ (`T[,]`) เหมาะกับข้อมูลแบบตารางหรือกริด ส่วนอาร์เรย์ซ้อน (`T[][]`) คืออาร์เรย์ของอาร์เรย์ที่แต่ละแถวยาวไม่เท่ากันได้

`List<T>` คืออาร์เรย์ที่ขยายขนาดได้เอง การเพิ่มท้ายเป็น O(1) แบบเฉลี่ย การเข้าถึงด้วยดัชนีเป็น O(1) แต่การแทรก ลบตรงกลาง หรือค้นหาค่าเป็น O(n) จึงเหมาะกับข้อมูลที่เพิ่มต่อท้ายแล้ววนอ่าน

2.3 สแตกและคิว

สแตกทำงานแบบ LIFO (เข้าทีหลังออกก่อน) มีการทำงาน Push และ Pop ที่เป็น O(1) เหมาะกับงานที่ซ้อนกันแล้วต้องย้อนกลับ เช่น Undo หรือหน้าจอที่เปิดทับกัน คิวทำงานแบบ FIFO (เข้าก่อนออกก่อน) มีการทำงาน Enqueue และ Dequeue ที่เป็น O(1) `Queue<T>` ของ .NET ใช้บัฟเฟอร์วงกลมจึงไม่ต้องเลื่อนข้อมูล คิวเหมาะกับงานที่ต้องทำตามลำดับที่เข้ามา

2.4 ตารางแฮช

ตารางแฮชเก็บข้อมูลเป็นคู่คีย์กับค่า โดยใช้ฟังก์ชันแฮชของคีย์หาตำแหน่งที่เก็บ ทำให้เพิ่ม ค้นหา และลบได้ใน O(1) โดยเฉลี่ย ใน C# คือ `Dictionary<K,V>` ส่วน

`HashSet<T>` เก็บเฉพาะคีย์ ใช้ตรวจว่ามีข้อมูลนั้นอยู่หรือไม่ใน O(1) และไม่เก็บค่าซ้ำ คีย์อาจเป็นสตริง enum struct อย่าง `Point` หรือ tuple ก็ได้

2.5 กราฟ ต้นไม้ และการค้นหาแนวกว้าง

กราฟประกอบด้วยจุดยอดและเส้นเชื่อม แทนในหน่วยความจำได้ด้วยเมทริกซ์ประชิด (ใช้พื้นที่ $O(V^2)$) หรือรายการประชิด (ใช้พื้นที่ $O(V + E)$) ซึ่งเหมาะกับกราฟที่มีเส้นเชื่อมน้อย ต้นไม้คือกราฟที่เชื่อมถึงกันทั้งหมดและไม่มีวงวน

การค้นหาแนวกว้าง (Breadth-First Search: BFS) เริ่มจากจุดต้นทางแล้วขยายออกทีละชั้นโดยใช้คิว บนกราฟที่ทุกเส้นมีต้นทุนเท่ากัน BFS จะได้ระยะทางที่สั้นที่สุดไปยังทุกจุดในเวลา $O(V + E)$ ถ้าเก็บจุดก่อนหน้าของแต่ละจุดไว้ก็ย้อนกลับเพื่อสร้างเส้นทางได้ การใช้ BFS บนกริดเพื่อหาพื้นที่ที่เดินถึงได้ทั้งหมดเรียกว่า Flood Fill

2.6 บิตมาสก์และการเรียงลำดับหลายคีย์

บิตมาสก์ใช้บิตแต่ละบิตของจำนวนเต็มแทนสมาชิกของเซตขนาดเล็ก การตรวจสมาชิก การรวม หรือการตรวจสับเซตทำได้ด้วยตัวดำเนินการระดับบิตใน $O(1)$ ส่วนการเรียงหลายคีย์คือเรียงด้วยคีย์หลักก่อนแล้วใช้คีย์รองเมื่อค่าเท่ากัน ใน C# ใช้ `OrderBy(...).ThenBy(...)` ซึ่งเป็นการเรียงแบบเสถียร (ข้อมูลที่เท่ากันคงลำดับเดิม) ใช้เวลา $O(n \log n)$

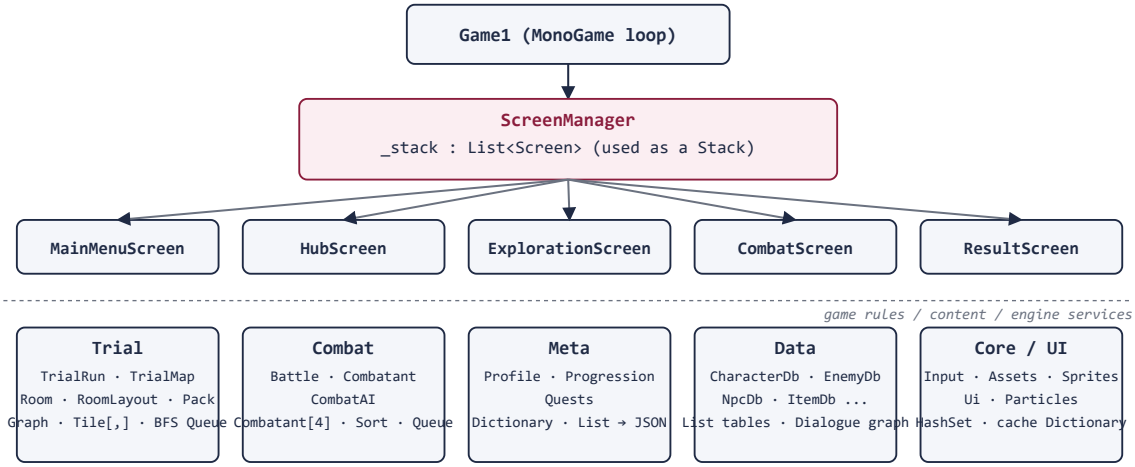
ตารางที่ 2.1 ความซับซ้อนของการทำงานหลัก (กรณีเฉลี่ย)

โครงสร้าง	เข้าถึง	ค้นหาค่า	เพิ่ม	ลบ
`T[]`, `T[,]`	O(1)	O(n)	-	-
`List<T>`	O(1)	O(n)	O(1) ท้ายลิสต์	O(n)
Stack / Queue	บนสุด/หัวคิว O(1)	O(n)	O(1)	O(1)
`Dictionary` / `HashSet`	O(1)	ตามคีย์ O(1)	O(1)	O(1)
บิตมาสก์	O(1)	O(1)	O(1)	O(1)

บทที่ 3
การออกแบบระบบ

3.1 โครงสร้างของโปรแกรม

โค้ดแบ่งเป็นส่วนตามหน้าที่ `Game1` คือลูปหลักของ MonoGame ที่เรียก `Update()` และ `Draw()` ทุกเฟรม `ScreenManager` จัดการหน้าจอต่าง ๆ ในรูปแบบสแตก ส่วนกฎของเกมแยกไว้นอกหน้าจอ ได้แก่ Trial (ด่านและห้อง) Combat (กฎการต่อสู้ที่ไม่มีส่วนวาดภาพ) Meta (ข้อมูลที่บันทึกไว้) และ Data (ข้อมูลเนื้อหาของเกม) การแยกแบบนี้ทำให้แต่ละส่วนเลือกโครงสร้างข้อมูลที่เหมาะกับงานของตัวเองได้



ภาพที่ 3.1 โครงสร้างของโปรแกรมและโครงสร้างข้อมูลหลักในแต่ละส่วน

3.2 เกณฑ์การเลือกโครงสร้างข้อมูล

กลุ่มพิจารณาจาก 4 เรื่อง คือ (1) ระบบนั้นเข้าถึงข้อมูลแบบไหน เช่น ตามดัชนี ตามคีย์ หรือตามลำดับที่เข้ามา (2) ขนาดของข้อมูลคงที่หรือเปลี่ยนได้ และมากน้อยแค่ไหน ถ้าข้อมูลมีไม่เกินสิบตัว การค้นหาเชิงเส้นก็เพียงพอ (3) โค้ดนั้นทำงานทุกเฟรมหรือทำครั้งเดียว และ (4) โครงสร้างนั้นตรงกับกฎในเอกสารออกแบบหรือไม่

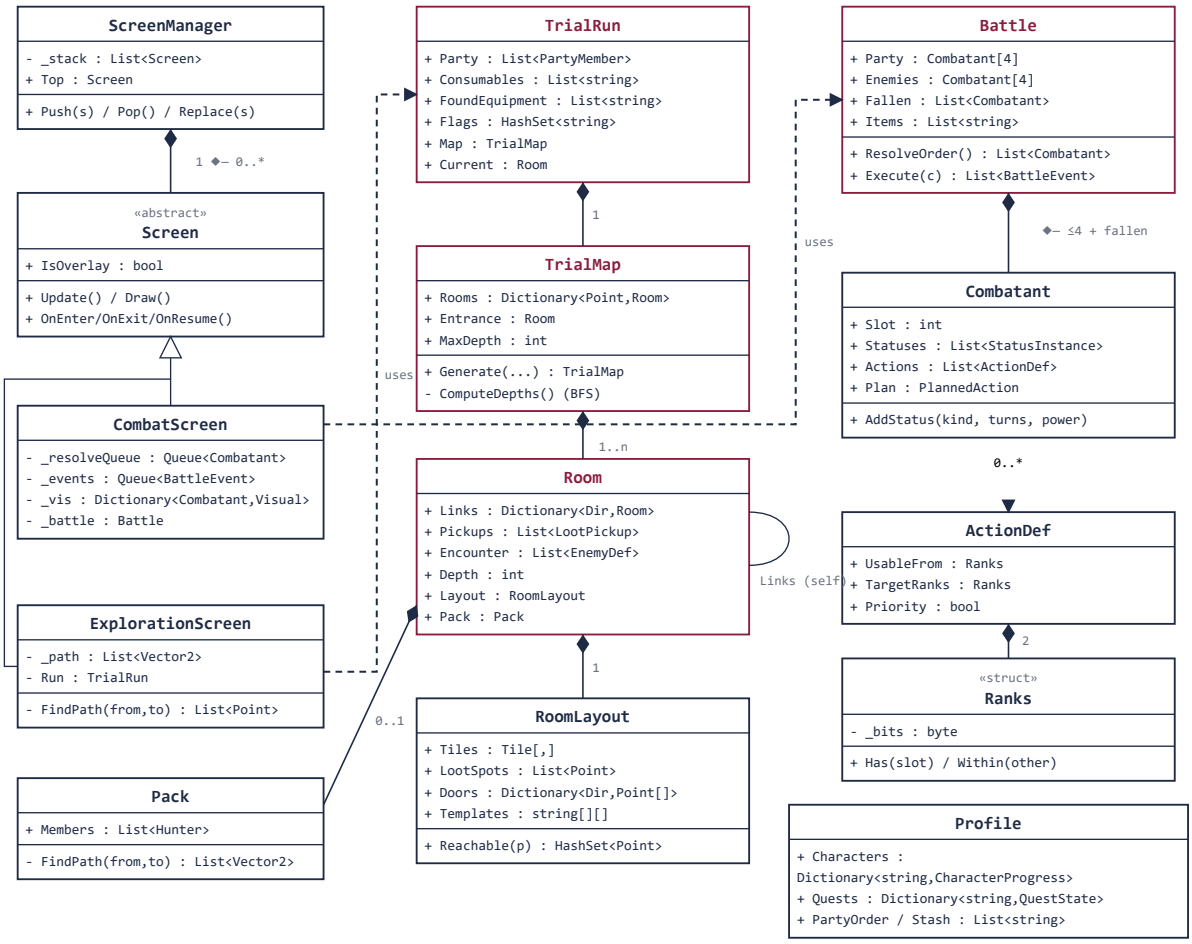
ตารางที่ 3.1 ระบบในเกมและโครงสร้างข้อมูลที่ใช้

ระบบ	โครงสร้างข้อมูล	หัวข้อ
การจัดการหน้าจอ	สแตก (`List<Screen>`)	4.1
แผนที่ Trial	กราฟ `Dictionary<Point,Room>` และ `Dictionary<Dir,Room>`, BFS	4.2

ผังห้อง	`Tile[,]`, `string[][]`, `HashSet<Point>`	4.3
การหาเส้นทาง	BFS ด้วย `Queue` และ `Dictionary`	4.4
แถวการรบ	`Combatant[4]`, บิตมาสก์ `Ranks`	4.5
ลำดับการกระทำ	การเรียงหลายคีย์, `Queue` สองชุด	4.6
สถานะและ AI	`List<StatusInstance>`, `List<(T, double)>`	4.7
เนื้อหาและบทสนทนา	`List`, กราฟมีทิศทาง	4.8
แคชและแฟล็ก	`Dictionary`, `HashSet`	4.9
การบันทึกเกม	`Dictionary` และ `List` แปลงเป็น JSON	4.10

3.3 แผนภาพคลาส

แผนภาพ UML ด้านล่างแสดงคลาสหลักและฟิลด์ที่เป็นโครงสร้างข้อมูล เส้นที่มีข้าวหลามตัดทึบคือความสัมพันธ์แบบ Composition (วัตถุหนึ่งเป็นเจ้าของอีกวัตถุ) เส้นประคือการเรียกใช้ และสามเหลี่ยมคือการสืบทอดคลาส

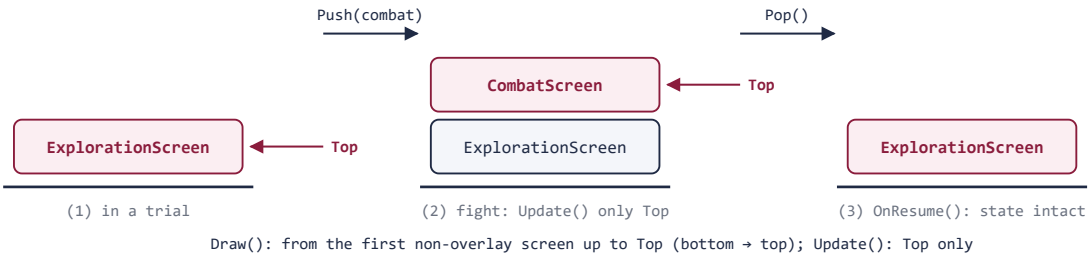


ภาพที่ 3.2 แผนภาพคลาส UML ของคลาสหลัก

บทที่ 4
การนำโครงสร้างข้อมูลไปใช้ในเกม

4.1 สแตกของหน้าจอ

เกมมีหลายหน้าจอ เช่น เมนู ฐาน การสำรวจ การต่อสู้ และหน้าสรุปผล เมื่อเกิดการต่อสู้ระหว่างสำรวจ หน้าจอสำรวจต้องหยุดไว้ทั้งหมด แล้วกลับมาที่สถานะเดิมหลังต่อสู้จบ `ScreenManager` จึงเก็บหน้าจอใน `List<Screen>` และใช้แบบสแตก โดยท้ายลิสต์คือหน้าจอบนสุด การต่อสู้ถูก Push ทับหน้าจอสำรวจ และ Pop ออกเมื่อจบ หน้าจอสำรวจยังอยู่ในหน่วยความจำครบจึงไม่ต้องสร้างใหม่



ภาพที่ 4.1 การ Push และ Pop หน้าจอการต่อสู้

```
public void PopInstant()
{
    if (_stack.Count > 0)
    {
        _stack[^1].OnExit();
        _stack.RemoveAt(_stack.Count - 1);   // remove the top: O(1)
    }
    Top?.OnResume();
}

public void Draw(SpriteBatch sb, GameTime gt)
{
    for (int i = FirstVisible(); i < _stack.Count; i++)   // bottom -> top
        _stack[i].Draw(sb, gt);
}
```

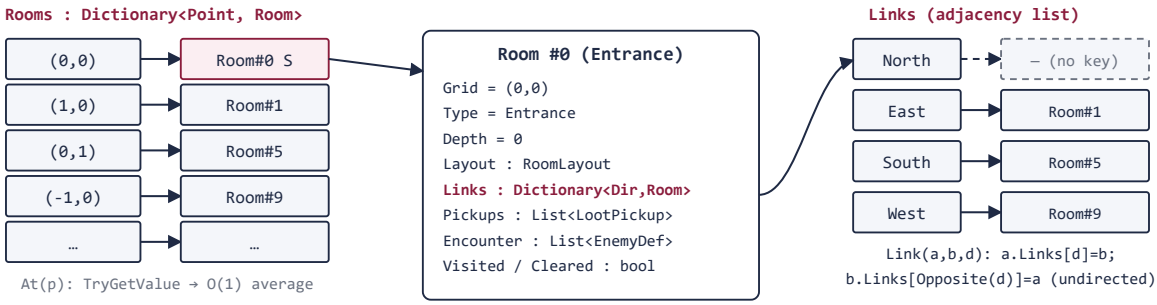
โค้ดที่ 4.1 การนำหน้าจอบนสุดออก และการวาดจากล่างขึ้นบน (Core/Screen.cs)

ที่ใช้ List แทน `Stack<T>` เพราะหน้าต่างหยุดเกมเป็นแบบโปร่งแสง ต้องวาดหน้าจอด้านล่างด้วย จึงต้องวนจากล่างขึ้นบนด้วยดัชนี ซึ่ง `Stack<T>` ทำไม่ได้ การ Push และ Pop เป็น O(1)

4.2 แผนที่ด่านแบบกราฟ

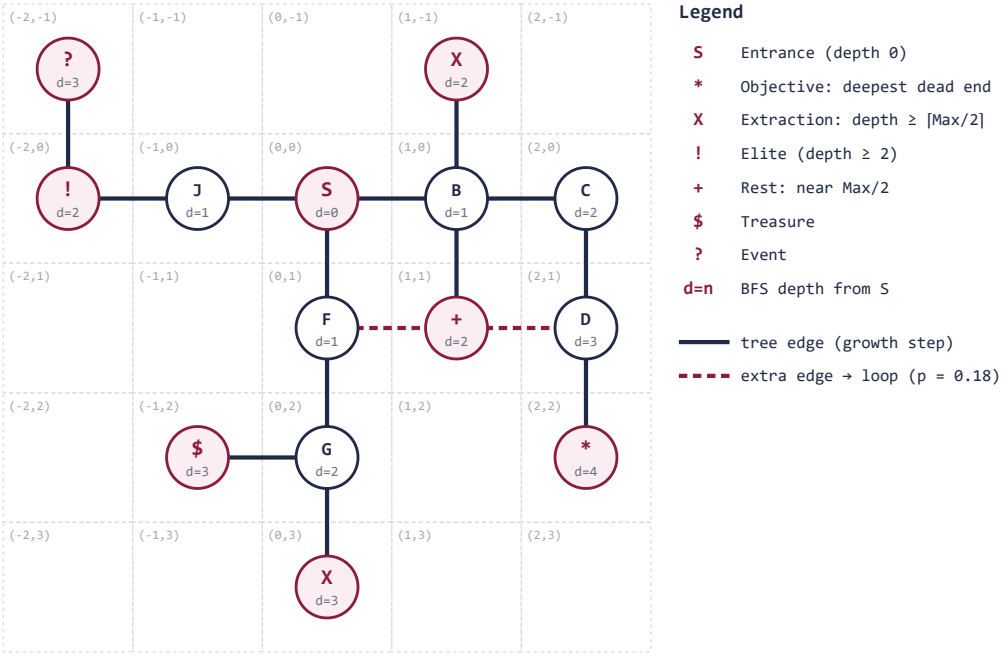
ทุกครั้งที่เข้า Trial เกมต้องสุ่มแผนที่ใหม่ ห้องทุกห้องต้องเดินถึงกันได้ มีทางแยกและทางวน และต้องวางห้องพิเศษตามระยะห่างจากทางเข้า เช่น ห้องเป้าหมายอยู่ปลายทางตันที่ลึกที่สุด และจุดถอนตัวต้องอยู่ห่างจากทางเข้าพอสมควร

แผนที่จึงเก็บเป็นกราฟไม่มีทิศทางสองระดับ ระดับแรกคือ `Dictionary<Point, Room> Rooms` ที่ใช้พิกัดกริดเป็นคีย์ ใช้หาห้องจากพิกัดหรือตรวจว่าช่องนั้นมีห้องแล้วหรือยังใน O(1) ระดับที่สองคือ `Dictionary<Dir, Room> Links` ในแต่ละห้อง ซึ่งเป็นรายการประชิดที่ใช้ทิศเป็นคีย์ ห้องหนึ่งมีทางเชื่อมได้ไม่เกิน 4 ทาง และคีย์ยังบอกด้วยว่าต้องเปิดประตูด้านไหน



ภาพที่ 4.2 ตารางแฮชของห้องและรายการประชิดภายในห้อง

การสร้างแผนที่เริ่มจากห้องทางเข้าที่ (0,0) แล้วสุ่มต่อห้องใหม่ออกจากห้องที่มีอยู่ทีละห้อง ห้องใหม่ทุกห้องเชื่อมกับห้องเดิมหนึ่งเส้น ผลที่ได้จึงเป็นต้นไม้ที่ทุกห้องเดินถึงกัน จากนั้นสุ่มเชื่อมห้องที่อยู่ติดกันเพิ่มด้วยโอกาส 18% เพื่อให้เกิดทางวน สุดท้ายใช้ BFS หาความลึก (จำนวนประตูจากทางเข้า) ของทุกห้อง แล้วใช้ค่านี้กำหนดประเภทห้องและความยาก



ภาพที่ 4.3 ตัวอย่างแผนที่ Trial เส้นทึบคือเส้นของต้นไม้ เส้นประคือทางวน และ d คือความลึกจาก BFS

```
private static void Link(Room a, Room b, Dir aToB)
{
    a.Links[aToB] = b;                          // undirected edge = two entries
    b.Links[RoomLayout.Opposite(aToB)] = a;
}

private void ComputeDepths()
{
    foreach (var r in Rooms.Values) r.Depth = -1;   // -1 = not visited
    var q = new Queue<Room>();
    Entrance.Depth = 0;
    q.Enqueue(Entrance);
    while (q.Count > 0)
    {
        var r = q.Dequeue();
        foreach (var n in r.Links.Values)
            if (n.Depth < 0) { n.Depth = r.Depth + 1; q.Enqueue(n); }
    }
    MaxDepth = Rooms.Values.Max(r => r.Depth);
}
```

โค้ดที่ 4.2 การเชื่อมห้องสองทางและการหาความลึกด้วย BFS (Trial/TrialMap.cs)

เหตุที่ไม่ใช้อาร์เรย์ `Room[9,9]` เพราะแผนที่มีเพียง 8–15 ห้องจาก 81 ช่อง และพิกัดติดลบได้ Dictionary เก็บเฉพาะห้องที่มีจริง เหตุที่ใช้ BFS แทน DFS เพราะ

ต้องการระยะทางที่สั้นที่สุดจากทางเข้า ซึ่ง DFS ไม่รับประกันเมื่อมีทางวน BFS ใช้เวลา O(V + E) และทำครั้งเดียวตอนเริ่ม Trial

4.3 ผังห้องแบบอาร์เรย์สองมิติ

ภายในห้องแบ่งเป็นช่องขนาด 48 พิกเซล เกมต้องรู้ว่าช่องไหนเดินได้ ช่องไหนเป็นกำแพงหรือเสา และจุดไหนวางของได้ การตรวจการชนเกิดทุกเฟรมกับตัวละครและศัตรูทุกตัว จึงต้องเร็ว

ผังห้องต้นแบบ 5 แบบเก็บเป็น `string[][] Templates` โดยแต่ละตัวอักษรแทนหนึ่งช่อง (`.` พื้น `o` เสา `L` จุดวางของ `F` วัตถุพิเศษ) ทำให้แก้ผังห้องได้ง่ายเหมือนวาดด้วยตัวอักษร เมื่อสร้างห้องจริง เกมแปลงเทมเพลตเป็น `Tile[21,12]` (สุ่มกลับซ้ายขวาได้) และเปิดประตูตามทิศใน `Links`

```
for (int iy = 0; iy < InnerRows; iy++)
    for (int ix = 0; ix < InnerCols; ix++)
    {
        char ch = src[iy][mirror ? InnerCols - 1 - ix : ix];   // mirror = read the
row backwards
        int x = ix + 1, y = iy + 1;                            // +1: outer wall ring
        layout.Tiles[x, y] = ch switch { 'o' => Tile.Pillar, ',' => Tile.Decor, _ =>
Tile.Floor };
        if (ch == 'L') layout.LootSpots.Add(new Point(x, y));
    }

public bool Solid(int x, int y) =>
    x < 0 || y < 0 || x >= Cols || y >= Rows || Tiles[x, y] == Tile.Wall || Tiles[x,
y] == Tile.Pillar;
```

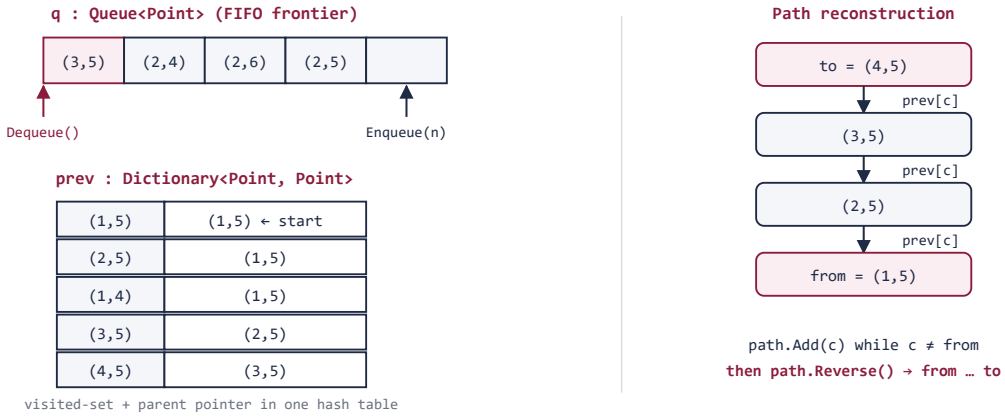
โค้ดที่ 4.3 การแปลงเทมเพลตเป็นอาร์เรย์สองมิติและการตรวจช่องทึบ (Trial/RoomLayout.cs)

ห้องมีขนาดคงที่และทุกช่องมีข้อมูล อาร์เรย์สองมิติจึงเหมาะที่สุด `Solid(x, y)` เป็น O(1) นอกจากนี้เมธอด `Reachable()` ยังใช้ BFS กับ `HashSet<Point>` หาทุกช่องที่เดินถึงได้จากประตู เกมใช้ผลนี้เลือกจุดเกิดของศัตรู (`Pack.Spawn`) และย้ายของที่ดรอปให้อยู่บนพื้นที่เดินถึงได้ (`TrialRun.ClampToFloor`)

4.4 การหาเส้นทางด้วย BFS

ผู้เล่นคลิกที่ใดก็ได้ในห้องเพื่อให้ตัวละครเดินไป และศัตรูที่ไล่ตามก็ต้องหาเส้นทางไปหาผู้เล่นใหม่เป็นระยะ ทั้งสองระบบใช้ BFS บนกริดของห้อง โดยใช้ `Queue<Point>` เก็บช่องที่รอสำรวจ และ `Dictionary<Point, Point> prev` เก็บช่องก่อน

หน้าของแต่ละช่อง ซึ่งใช้แทนเซตของช่องที่เยี่ยมแล้วไปในตัว เมื่อถึงปลายทางก็ย้อนตาม prev กลับไปต้นทางแล้วกลับลำดับลิสต์



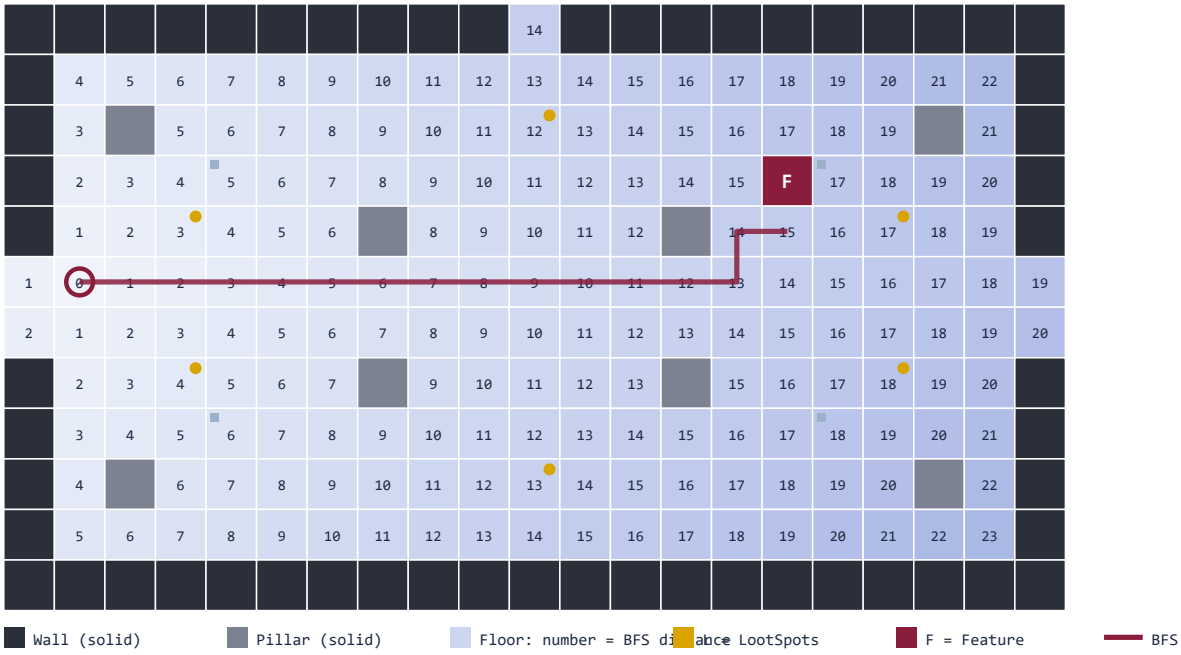
ภาพที่ 4.4 คิว ตารางช่องก่อนหน้า และการย้อนรอยสร้างเส้นทาง

```
private List<Point> FindPath(Point from, Point to)
{
    if (from == to) return new List<Point> { to };
    var prev = new Dictionary<Point, Point> { [from] = from };
    var q = new Queue<Point>();
    q.Enqueue(from);
    while (q.Count > 0)
    {
        var p = q.Dequeue();
        if (p == to) break;                                  // shortest path found
        foreach (var d in RoomLayout.AllDirs)
        {
            var s = RoomLayout.Step(d);
            var n = new Point(p.X + s.X, p.Y + s.Y);
            if (prev.ContainsKey(n) || !Walkable(n)) continue;
            prev[n] = p;
            q.Enqueue(n);
        }
    }
    if (!prev.ContainsKey(to)) return null;                  // unreachable
    var path = new List<Point>();
    for (var c = to; c != from; c = prev[c]) path.Add(c);
    path.Reverse();
    return path;
}
```

โค้ดที่ 4.4 การหาเส้นทางด้วย BFS (Screens/ExplorationScreen.cs)

ภาพที่ 4.5 แสดงผลการทำงานบนผังห้อง Plaza จริง ตัวเลขในแต่ละช่องคือระยะทาง BFS จากประตูด้านซ้าย มีช่องที่เดินถึงได้ 186 ช่อง และเส้นทางไปยังหน้า

วัตถุพิเศษยาว 15 ก้าว หลังได้เส้นทางแล้ว เกมจะตัดจุดที่เดินเป็นเส้นตรงถึงได้ออก เพื่อให้ตัวละครเดินตัดมุมแทนการเดินซิกแซก

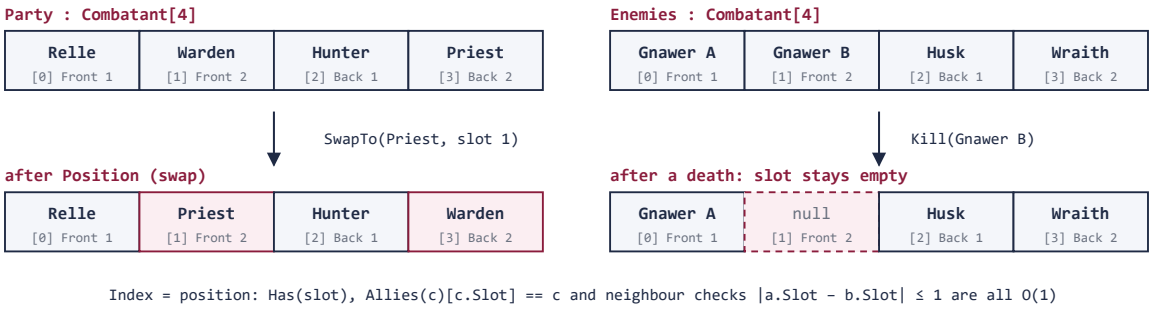


ภาพที่ 4.5 ผังห้อง Plaza ใน Tile[21,12] พร้อมระยะทาง BFS และเส้นทางที่สั้นที่สุด

กลุ่มเลือก BFS แทน A* เพราะห้องมีเพียง 252 ช่องและทุกก้าวมีต้นทุนเท่ากัน BFS จึงได้เส้นทางที่สั้นที่สุดเหมือนกัน โค้ดสั้นกว่า และไม่ต้องใช้คิวลำดับความสำคัญ ความซับซ้อนคือ O(W × H) เมื่อ W × H คือจำนวนช่องในห้อง

4.5 แถวการรบและบิตมาสก์ของตำแหน่ง

ตามเอกสารออกแบบ แต่ละฝ่ายมี 4 ตำแหน่ง ตัวละครสลับตำแหน่งได้ และเมื่อมีตัวตาย ตำแหน่งนั้นจะว่างโดยไม่มีใครเลื่อนขึ้นมาแทน `Battle` จึงใช้อาร์เรย์ `Combatant[4]` สำหรับแต่ละฝ่าย ดัชนีคือหมายเลขตำแหน่ง (0 = Front 1 ถึง 3 = Back 2) และช่องว่างมีค่า `null` ตัวที่ตายแล้วย้ายไปเก็บใน `List<Combatant> Fallen`



ภาพที่ 4.6 อาร์เรย์แถวการรบ การสลับตำแหน่ง และช่องว่างเมื่อมีตัวตาย

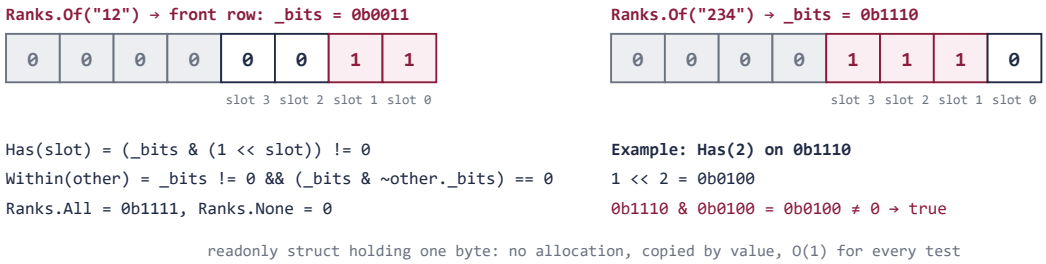
```
private void SwapTo(Combatant c, int slot, List<BattleEvent> ev)
{
    var side = Allies(c);
    int from = c.Slot;
    var other = side[slot];        // may be null (empty slot)
    side[slot] = c;
    side[from] = other;
    c.Slot = slot;
    if (other != null) other.Slot = from;
}

private void Kill(Combatant t, List<BattleEvent> ev)
{
    var side = Allies(t);
    if (side[t.Slot] == t) side[t.Slot] = null;   // the position becomes empty
    Fallen.Add(t);
}
```

โค้ดที่ 4.5 การสลับตำแหน่งและการตาย (Combat/Battle.cs)

ถ้าใช้ List แล้วลบตัวที่ตายออก ตัวที่อยู่ข้างหลังจะเลื่อนขึ้นมาเอง ซึ่งผิดกฎของเกม อาร์เรย์ที่มีช่อง `null` จึงตรงกว่า และการหาว่าใครอยู่ตำแหน่งไหนหรือการสลับตำแหน่งเป็น O(1) ข้อเสียคือทุกครั้งที่วนต้องตรวจ `null`

สกิลแต่ละอย่างกำหนดว่าใช้ได้จากตำแหน่งไหนและโจมตีตำแหน่งไหนได้ ข้อมูลนี้เก็บในโครงสร้าง `Ranks` ซึ่งเป็น struct ที่มี `byte` เพียงตัวเดียว บิตที่ 0–3 แทนตำแหน่งทั้ง 4 การตรวจว่ามีตำแหน่งนั้นหรือไม่ใช้ AND กับ `1 << slot` และการตรวจสับเซตใช้ `(a & ~b) == 0` ทุกการทำงานเป็น O(1) และไม่ต้องจองหน่วยความจำ



ภาพที่ 4.7 การเก็บเซตของตำแหน่งด้วยบิตมาสก์

```
public readonly struct Ranks
{
    private readonly byte _bits;

    public static Ranks Of(string ranks)              // "12" -> 0b0011
    {
        byte b = 0;
        foreach (char ch in ranks)
            if (ch >= '1' && ch <= '4') b |= (byte)(1 << (ch - '1'));
        return new Ranks(b);
    }

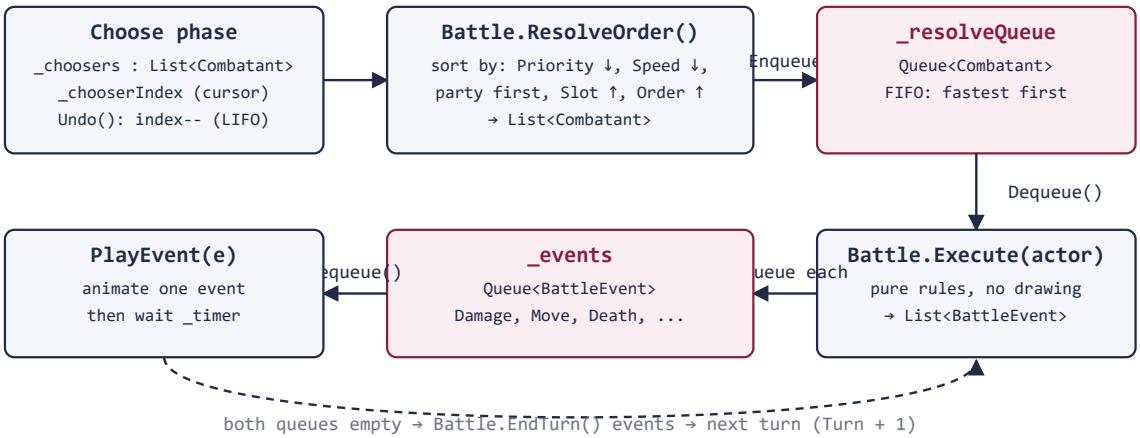
    public bool Has(int slot) => slot >= 0 && slot < 4 && (_bits & (1 << slot)) != 0;
    public bool Within(Ranks other) => _bits != 0 && (_bits & ~other._bits) == 0;
}
```

โค้ดที่ 4.6 โครงสร้าง Ranks (Data/ActionDef.cs)

4.6 ลำดับการกระทำและคิวเหตุการณ์

ในแต่ละเทิร์น ผู้เล่นเลือกการกระทำให้ทุกคน (กดย้อนกลับได้) จากนั้นการกระทำทั้งหมดทำงานตามลำดับ คือการกระทำแบบ Priority ก่อน แล้วเรียงตามความเร็วจากมากไปน้อย ถ้าเท่ากันให้ฝ่ายผู้เล่นและตำแหน่งหน้าได้ก่อน กฎคำนวณผลได้ทันที แต่ภาพต้องแสดงทีละขั้น

ช่วงเลือกการกระทำใช้ `List<Combatant>` กับตัวชี้ตำแหน่ง การกดย้อนกลับคือลดตัวชี้ลงหนึ่ง ซึ่งทำงานแบบสแตก เมื่อเลือกครบ `ResolveOrder()` จะเรียงด้วยหลายคีย์แล้วใส่ลง `Queue<Combatant>` เกมดึงผู้กระทำออกทีละตัว `Battle.Execute()` คืนรายการเหตุการณ์ (ความเสียหาย การเคลื่อนที่ การตาย) ซึ่งถูกใส่ลง `Queue<BattleEvent>` อีกชุดเพื่อเล่นแอนิเมชันทีละรายการ



ภาพที่ 4.8 ลำดับการทำงานในหนึ่งเทิร์น

```
public List<Combatant> ResolveOrder() =>
    Living.Where(c => c.Plan != null)
        .OrderByDescending(c => c.Plan.Action.Priority)
        .ThenByDescending(c => c.EffectiveSpeed)
        .ThenBy(c => c.IsParty ? 0 : 1)
        .ThenBy(c => c.Slot)
        .ThenBy(c => c.Order)          // spawn order: no ties left
        .ToList();

// CombatScreen
if (_events.Count > 0) _timer = PlayEvent(_events.Dequeue());
else if (_resolveQueue.Count > 0)
{
    var actor = _resolveQueue.Dequeue();
    foreach (var e in _battle.Execute(actor)) _events.Enqueue(e);
}
```

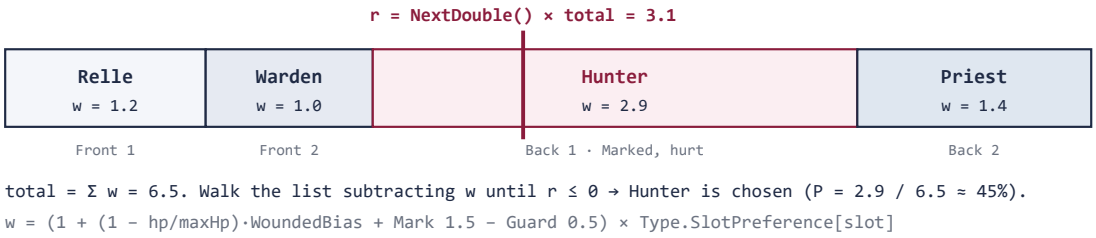
โค้ดที่ 4.7 การเรียงหลายคีย์และการใช้คิวสองชุด (Combat/Battle.cs, Screens/CombatScreen.cs)

มีตัวละครในการต่อสู้ไม่เกิน 8 ตัว การเรียงครั้งเดียว O(n log n) จึงพอ และยังใช้แสดงแถบลำดับเทิร์นบนจอได้ด้วย การเรียงแบบเสถียรและคีย์สุดท้ายที่ไม่ซ้ำทำให้ลำดับออกมาเหมือนเดิมทุกครั้ง ส่วนคิวสองชุดทำให้ส่วนกฎกับส่วนแอนิเมชันแยกกันและเหตุการณ์แสดงตามลำดับที่เกิดจริง

4.7 สถานะผิดปกติและ AI ของศัตรู

ตัวละครแต่ละตัวเก็บสถานะผิดปกติ (เช่น Guard, Mark, Slow) ใน `List<StatusInstance>` ถ้าได้รับสถานะชนิดที่มีอยู่แล้ว เกมจะรวมกับรายการเดิมโดยเก็บค่าที่มากกว่า แทนการเพิ่มรายการซ้ำ สถานะมีเพียง 8 ชนิด การค้นหาเชิงเส้นจึงเร็วพอ และ `RemoveAll()` ลบสถานะตามเงื่อนไขได้ในรอบเดียว

ศัตรูเลือกเป้าหมายแบบสุ่มถ่วงน้ำหนัก (Roulette Wheel) โดยสร้าง `List<(Combatant, double)>` ของเป้าหมายพร้อมน้ำหนัก ตัวที่เลือดน้อยหรือติด Mark มีน้ำหนักมากขึ้น และคูณด้วยค่าความชอบตำแหน่งของศัตรูแต่ละประเภท (`float[4]`) จากนั้นสุ่มตัวเลขระหว่าง 0 ถึงผลรวมน้ำหนัก แล้วลบน้ำหนักทีละตัวจนค่าน้อยกว่าหรือเท่ากับศูนย์ ใช้เวลา O(n) ซึ่งพอเพราะมีเป้าหมายไม่เกิน 4 ตัว

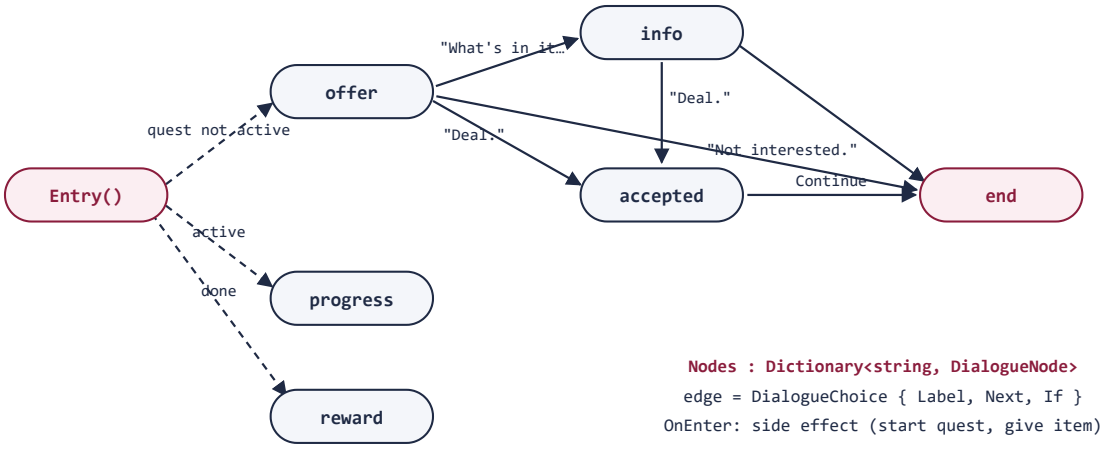


ภาพที่ 4.9 การสุ่มเป้าหมายแบบถ่วงน้ำหนัก

4.8 ข้อมูลเนื้อหาและกราฟบทสนทนา

ข้อมูลตัวละคร ศัตรู ไอเทม และด่าน เก็บเป็น `List` ในคลาส `CharacterDb`, `EnemyDb`, `ItemDb` เป็นต้น เพราะส่วนใหญ่ถูกวนอ่านทั้งชุด เช่น ตอนแสดงเมนูหรือสุ่มศัตรู และลำดับในลิสต์คือลำดับที่แสดงผล

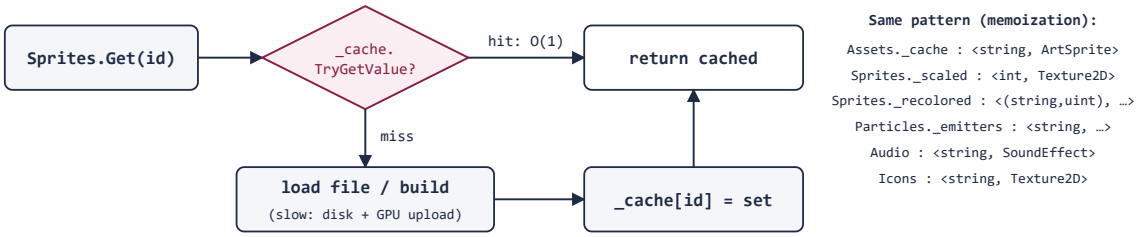
บทสนทนาของ NPC เป็นกราฟมีทิศทาง เก็บใน `Dictionary<string, DialogueNode>` แต่ละโหนดมีข้อความและรายการตัวเลือก ตัวเลือกแต่ละตัวคือเส้นเชื่อมที่ชี้ไปโหนดถัดไปด้วยชื่อ และมีเงื่อนไขซ่อนตัวเลือกได้ การอ้างอิงโหนดด้วยชื่อทำให้วนกลับไปโหนดเดิมได้ง่าย และหาโหนดถัดไปได้ใน O(1)



ภาพที่ 4.10 กราฟบทสนทนาของ NPC ชื่อ Maren

4.9 แคชทรัพยากรและแฟล็ก

การโหลดภาพ ตัดเฟรม ย่อขยาย หรือเปลี่ยนสี sprite ใช้เวลานาน เกมจึงเก็บผลไว้ใน `Dictionary` เมื่อเรียกครั้งแรกจะโหลดแล้วเก็บไว้ ครั้งต่อไปดึงจากตารางได้ทันที คีย์มีหลายแบบ เช่น ชื่อไฟล์ (`string`) ความสูงของภาพ (`int`) หรือ tuple (`string, uint`) สำหรับ sprite ที่เปลี่ยนสีแล้ว ซึ่งไม่ต้องต่อสตริงเป็นคีย์



ภาพที่ 4.11 การแคชทรัพยากรด้วย Dictionary

สำหรับข้อมูลแบบ "เกิดขึ้นแล้วหรือยัง" เกมใช้ `HashSet` เช่น `TrialRun.Flags` เก็บเหตุการณ์ที่เกิดใน Trial นี้เพื่อใช้เป็นเงื่อนไขของบทสนทนา และหน้าจอการต่อสู้ใช้ `HashSet<Combatant>` เก็บตัวที่ถูกเล็งอยู่ ซึ่งต้องตรวจทุกตัวทุกเฟรม

4.10 การบันทึกเกม

ข้อมูลที่คงอยู่ข้าม Trial อยู่ในคลาส `Profile` และบันทึกเป็นไฟล์ JSON ระดับอัปเกรดและอุปกรณ์ของตัวละครเก็บใน `Dictionary<string, CharacterProgress>` โดยใช้รหัสตัวละครเป็นคีย์ ถ้าเพิ่มตัวละครใหม่ภายหลังไฟล์บันทึกเก่าก็ยังใช้ได้ ส่วนลำดับทีมและคลังอุปกรณ์เก็บใน `List<string>` เพราะลำดับมีความหมาย ทั้ง Dictionary และ List แปลงเป็น JSON ได้โดยตรง (ตัวอย่างในภาคผนวก ค) ของที่ได้ระหว่าง Trial เก็บแยกใน `TrialRun` และย้ายเข้า `Profile` เฉพาะเมื่อถอนตัวสำเร็จ

4.11 โครงสร้างข้อมูลที่วางแผนจะใช้

ตารางที่ 4.1 โครงสร้างข้อมูลที่วางแผนจะนำมาใช้

โครงสร้างข้อมูล	ใช้กับ และเหตุผล
`PriorityQueue` และ A*	ถ้าเพิ่มพื้นที่ที่เดินช้า (เช่น โคลน) กราฟจะมีน้ำหนัก BFS จะไม่ได้เส้นทางที่ดีที่สุดอีก ต้องเปลี่ยนเป็น A*
Object Pool (`Stack<T>`)	นำวัตถุชั่วคราว เช่น ตัวเลขดาเมจ กลับมาใช้ใหม่ ลดงานของ Garbage Collector

บัฟเฟอร์วงกลม	บันทึกการต่อสู้ตอนนี้ใช้ `List.RemoveAt(0)` ซึ่งเป็น O(n) บัฟเฟอร์วงกลมทำได้ใน O(1)
Spatial Hash	ถ้าในห้องมีศัตรูมาก ใช้แบ่งพื้นที่เป็นช่องเพื่อตรวจการชนเฉพาะตัวที่อยู่ใกล้กัน
DAG และ Topological Sort	เงื่อนไขการปลดล็อกพื้นที่และภารกิจ เพื่อหาลำดับที่ถูกต้องและตรวจว่าไม่มีเงื่อนไขวนกัน

บทที่ 5
สรุปผลและข้อเสนอแนะ

5.1 สรุปผล

เกม Aetheris ใช้โครงสร้างข้อมูลพื้นฐานครบทุกกลุ่มในรายวิชา และเลือกตามลักษณะการใช้งานของแต่ละระบบ ดังตารางที่ 5.1 ระบบที่ทำงานทุกเฟรม เช่น การตรวจการชนและการเล็งเป้า ใช้การทำงานแบบ O(1) ส่วนงานที่หนักกว่า เช่น BFS และการสร้างแผนที่ ทำเป็นครั้งคราวกับข้อมูลขนาดเล็ก

ตารางที่ 5.1 สรุปโครงสร้างข้อมูลที่ใช้ในเกม

โครงสร้างข้อมูล	**การใช้งานในเกม**
อาร์เรย์ / อาร์เรย์สองมิติ	แถวการรบ 4 ช่อง ผังห้อง 21×12
List	ข้อมูลเนื้อหา สถานะผิดปกติ ของดรอป
สแตก	หน้าจอ การย้อนกลับตอนเลือกการกระทำ
คิว	BFS ลำดับผู้กระทำ ลำดับเหตุการณ์
Dictionary / HashSet	ห้องตามพิกัด รายการประชิด แคช บทสนทนา แฟล็ก ไฟล์บันทึก
กราฟ / ต้นไม้	แผนที่ Trial กราฟบทสนทนา
บิตมาสก์	ตำแหน่งที่สกิลใช้ได้และโจมตีได้

5.2 ปัญหาที่พบ

1. การใช้ `null` แทนช่องว่างในแถวการรบทำให้ต้องตรวจ null ทุกครั้งที่วน กลุ่มจึงรวมการกรองไว้ใน property เช่น `LivingParty`
2. คำสั่ง LINQ เช่น `Where` และ `OrderBy` สร้างวัตถุใหม่ทุกครั้ง ใช้ได้ในโค้ดที่ทำงานนาน ๆ ครั้ง แต่ต้องระวังในโค้ดที่ทำงานทุกเฟรม
3. .NET ไม่รับประกันลำดับการวนของ `Dictionary` การสร้างแผนที่ที่ต้องได้ผลเหมือนเดิมเมื่อใช้ seed เดิมจึงต้องหลีกเลี่ยงการลบข้อมูลระหว่างสร้าง

5.3 ข้อเสนอแนะ

ในการพัฒนาต่อ กลุ่มควรเปลี่ยนไปใช้ A* เมื่อมีพื้นที่ที่มีต้นทุนการเดินต่างกัน ทำ Object Pool สำหรับวัตถุชั่วคราวในการต่อสู้ และสร้างดัชนีแบบ Dictionary ให้

ข้อมูลเนื้อหาเมื่อจำนวนข้อมูลเพิ่มขึ้น

บรรณานุกรม

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Microsoft. (2025). *System.Collections.Generic Namespace*. Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.collections.generic

MonoGame Foundation. (2024). *MonoGame Documentation*. https://docs.monogame.net

Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning. https://gameprogrammingpatterns.com

Patel, A. (2014). *Introduction to the A* Algorithm*. Red Blob Games. https://www.redblobgames.com/pathfinding/a-star/introduction.html

ภาคผนวก ก
โค้ดการสร้างแผนที่

```
public static TrialMap Generate(TrialDef trial, Random rng, bool withNpcRoom = false,
int objectiveRooms = 0)
{
    var map = new TrialMap();
    var start = new Room { Id = 0, Grid = Point.Zero, Type = RoomType.Entrance };
    map.Rooms[start.Grid] = start;
    map.Entrance = start;
    var order = new List<Room> { start };

    // Grow a tree: attach new rooms to random existing ones, biased toward recent
rooms
    int guard = 0;
    while (map.Rooms.Count < trial.RoomCount && guard++ < 5000)
    {
        var from = order[Math.Max(0, order.Count - 1 - (int)
(Math.Pow(rng.NextDouble(), 2) * order.Count))];
        var dir = RoomLayout.AllDirs[rng.Next(4)];
        var step = RoomLayout.Step(dir);
        var p = new Point(from.Grid.X + step.X, from.Grid.Y + step.Y);
        if (Math.Abs(p.X) > GridRadius || Math.Abs(p.Y) > GridRadius ||
map.Rooms.ContainsKey(p)) continue;
        if (NeighbourCount(map, p) > 2) continue;          // avoid tight 2x2 clumps

        var room = new Room { Id = map.Rooms.Count, Grid = p, Type = RoomType.Normal
};
        map.Rooms[p] = room;
        Link(from, room, dir);
        order.Add(room);
    }

    // A few extra connections create loops (alternate routes).
    foreach (var r in map.Rooms.Values)
        foreach (var d in new[] { Dir.East, Dir.South })
        {
            var s = RoomLayout.Step(d);
            var n = map.At(new Point(r.Grid.X + s.X, r.Grid.Y + s.Y));
            if (n != null && !r.Links.ContainsKey(d) && rng.NextDouble() < 0.18)
Link(r, n, d);
        }

    map.ComputeDepths();
    map.AssignTypes(trial, rng, withNpcRoom, objectiveRooms);

    foreach (var r in map.Rooms.Values)
    {
        r.Template = rng.Next(RoomLayout.Templates.Length);
        r.Mirror = rng.Next(2) == 0;
```

```
        r.Layout = RoomLayout.Build(r.Template, r.Mirror, r.Links.Keys);   // doors =
keys of Links
    }
    return map;
}
```

โค้ด ก.1 TrialMap.Generate (Trial/TrialMap.cs)

ภาคผนวก ข
รายการโครงสร้างข้อมูลในโปรเจกต์

ตารางที่ ข.1 ฟิลด์ที่เป็นโครงสร้างข้อมูลในโค้ด

ไฟล์	ชื่อ : ชนิด	หน้าที่
Core/Screen.cs	`_stack : List<Screen>`	สแตกของหน้าจอ
Core/Assets.cs, Sprites.cs, Audio.cs	`Dictionary<string, ...>`, `Dictionary<(string,uint), SpriteSet>`	แคชภาพและเสียง
Trial/TrialMap.cs	`Rooms : Dictionary<Point, Room>`, `Queue<Room>`	กราฟห้องและ BFS
Trial/Room.cs	`Links : Dictionary<Dir, Room>`, `Pickups : List<LootPickup>`	รายการประชิด ของดรอป
Trial/RoomLayout.cs	`Tiles : Tile[,]`, `Templates : string[][]`, `HashSet<Point>`	ผังห้อง และ Flood Fill
Trial/Pack.cs	`Members : List<Hunter>`, `Queue<Point>`	ศัตรูที่ไล่ตามและ BFS
Trial/TrialRun.cs	`Party : List<PartyMember>`, `Flags : HashSet<string>`	สถานะของ Trial
Combat/Battle.cs	`Party, Enemies : Combatant[4]`, `Fallen : List<Combatant>`	แถวการรบ
Combat/Combatant.cs	`Statuses : List<StatusInstance>`	สถานะผิดปกติ
Combat/CombatAI.cs	`List<(T, double)>`	การสุ่มถ่วงน้ำหนัก
Data/ActionDef.cs	`Ranks` (บิตมาสก์ `byte`)	ตำแหน่งของสกิล
Data/NpcDb.cs	`Nodes : Dictionary<string, DialogueNode>`	กราฟบทสนทนา
Meta/Profile.cs	`Dictionary<string, CharacterProgress>`, `List<string>`	ไฟล์บันทึก
Screens/CombatScreen.cs	`Queue<Combatant>`, `Queue<BattleEvent>`, `HashSet<Combatant>`	คิวเทิร์น การเล็งเป้า

ภาคผนวก ค
ตัวอย่างไฟล์บันทึกเกม

ตัวอย่างไฟล์ `profile.json` (ค่าเป็นค่าสมมติ) Dictionary กลายเป็น JSON object ที่ใช้รหัสเป็นชื่อฟิลด์ และ List กลายเป็น JSON array

```
{
  "SecuredShards": 340,
  "Characters": {
    "relle":  { "HpLevel": 2, "DamageLevel": 1, "SpeedLevel": 0, "Weapon":
"rusted_blade", "Charm": null },
    "warden": { "HpLevel": 1, "DamageLevel": 0, "SpeedLevel": 0, "Weapon": null,
"Charm": "pale_locket" }
  },
  "Stash": [ "butchers_hatchet" ],
  "PartyOrder": [ "warden", "relle", "hunter", "priest" ],
  "Quests": { "maren_hunt": { "Progress": 8, "Claimed": true } },
  "Extractions": 4,
  "Defeats": 3
}
```

โค้ด ค.1 ตัวอย่างไฟล์ profile.json